

8 Puzzles A* (A Star) using Manhattan Search

```
import heapq

GOAL = (1, 2, 3, 8, 0, 4, 7, 6, 5)
POS = {n: divmod(i, 3) for i, n in enumerate(GOAL) if n}

def manhattan(s):
    return sum(
        abs(i // 3 - POS[n][0]) + abs(i % 3 - POS[n][1])
        for i, n in enumerate(s) if n
    )

def neighbors(s):
    z = s.index(0)
    r, c = divmod(z, 3)

    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = r + dr, c + dc

        if 0 <= nr < 3 and 0 <= nc < 3:
            n = list(s)
            x = nr * 3 + nc
            n[z], n[x] = n[x], n[z]
            yield tuple(n)

def inversions(s):
    a = [x for x in s if x]
    return sum(a[i] > a[j]
               for i in range(8)
               for j in range(i + 1, 8))

def solvable(s):
    return inversions(s) % 2 == inversions(GOAL) % 2

def astar(start):
    pq = [(manhattan(start), 0, start, [start])]
    seen = {start: 0}
    while pq:
        f, g, s, path = heapq.heappop(pq)
```

```

        if s == GOAL:
            return path

    for n in neighbors(s):
        ng = g + 1

        if n not in seen or ng < seen[n]:
            seen[n] = ng
            heapq.heappush(
                pq, (ng + manhattan(n), ng, n, path + [n])
            )

def print_board(s):
    for i in range(0, 9, 3):
        print(" ".join(str(x) if x else " " for x in s[i:i + 3]))
    print("-" * 9)

while True:
    try:
        s = tuple(map(int, input("Enter 9 numbers: ").split()))

        if len(s) == 9 and sorted(s) == list(range(9)):
            break

        print("Error: Enter numbers 0 to 8 exactly once.")

    except ValueError:
        print("Error: Enter integers only.")

print("\nInitial State:")
print_board(s)

print("Target Goal State:")
print_board(GOAL)

if not solvable(s):
    print("This puzzle configuration is UNSOLVABLE.")
else:
    print("Solvability check passed! Solving using A* Search "
          "(Manhattan Distance)...\n")

```

```

solution = astar(s)
if solution:
    print(f"Solution found in {len(solution) - 1} steps!\n")

    for step, state in enumerate(solution):
        print(f"Step {step}:")
        print_board(state)
else:
    print("No solution found.")

```

```

PS C:\Users\Student\Desktop\1WA24CS203> & C:\Users\Student\AppData\Local\Python\
les_Astar_Heuristic.py
Enter 9 numbers: 2 8 3 1 6 4 7 0 5

```

Initial State:

```

2 8 3
1 6 4
7 5
-----

```

Goal State:

```

1 2 3
8 4
7 6 5
-----

```

Solvability check passed! Solving using A* Search...

Solution found in 5 steps!

Step 0:

```

2 8 3
1 6 4
7 5
-----

```

Step 1:

```

2 8 3
1 4
7 6 5
-----

```

Step 2:

```

2 3
1 8 4
7 6 5
-----

```

Step 3:

```

2 3
1 8 4
7 6 5
-----

```

Step 4:

```

1 2 3
8 4
7 6 5
-----

```

Step 5:

```

1 2 3
8 4
7 6 5
-----

```